IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Buscar buenos hiperparámetros

GridSearchCV, RandomizedSearchCV y una nota de validación honesta

Sprint 8 · S08_05 · 40 min en casa (ejercicios 1 a 4 y mini-reto) · guía del paso 3 del reto

TU TARDE DEL 11/11 (CASA)

Qué necesitas para el paso 3 del reto

Parte del cuaderno	Qué haces	Tiempo	Para qué
Sección 1	Conceptos y dibujo de rejilla frente a azar	lectura	Entender lo que viene
Secciones 2 y 3	Ejercicio 1 (rejilla con un bosque) y ejercicio 2 (aleatorio con LightGBM)	≈ 25 min	**Imprescindible** para el reto
Secciones 4 a 7	Ejercicios 3 y 4, mini-reto y preguntas	≈ 15 min	Completan el tema y **entran en el test**

Paso 3 del reto «Duelo de modelos»

Registrar un boosting y una SVM, y **buscar hiperparámetros en dos familias**, solo con train y validación cruzada, sin pasar de **5 minutos** cada búsqueda.

Hazlo en casa la tarde del miércoles 11/11, antes de la sesión del jueves.

SECCIÓN 1 · CONCEPTOS

Parámetros frente a hiperparámetros

	Parámetros	Hiperparámetros
Quién los fija	El **modelo**, al entrenar	**Nosotros**, antes de entrenar
Ejemplos	El umbral que elige un árbol	**max_depth**, **C**, **learning_rate**, **k**

```
antelacion = [5, 10, 20, 40, 60, 90, 120, 150]   # días; 0 las 4 primeras, 1 las 4 últimas
arbol = DecisionTreeClassifier(max_depth=2)      # hiperparámetro: lo fijo yo
arbol.fit(X, y); arbol.tree_.threshold[0]        # 50.0: parámetro, lo aprende
```

Los hiperparámetros son los botones del horno: los giras tú antes de cocinar. Los parámetros salen de la cocción.

SECCIÓN 1 · QUÉ HACE UN BUSCADOR

Un buscador gira los botones por ti

1 · Elige

Una combinación de hiperparámetros.

2 · Evalúa

Con **validación cruzada**, solo sobre train.

3 · Repite

Con todas las combinaciones que le des.

4 · Reentrena

refit=True: la mejor, con todo train.

Rejilla	Combinaciones	Pliegues	Entrenamientos
max_depth [2, 4, 6] × **min_samples_leaf** [1, 5] × **criterion** [gini, entropy]	3 × 2 × 2 = 12	4	12 × 4 = 48, **+ 1** del reentrenado = 49

Cada hiperparámetro nuevo multiplica el coste: por eso la rejilla se dispara.

REJILLA · EJERCICIO 1 (CASA)

GridSearchCV: todas las combinaciones

```
pipe = Pipeline([("esc", StandardScaler()), ("rf", RandomForestClassifier())])
GridSearchCV(pipe, {"max_depth": [2, 4]}, cv=3).fit(X, y)
ValueError: Invalid parameter 'max_depth' for estimator Pipeline(...)
GridSearchCV(pipe, {"rf__max_depth": [2, 4]}, cv=3).fit(X, y)
```

El prefijo es el nombre del paso

Va pegado con **dos guiones bajos**: **rf__max_depth**. Olvidarlo es el error más frecuente.

En el reto cambia el nombre

hacer_pipe llama **clf** a la parte del modelo: ahí es **clf__max_depth**.

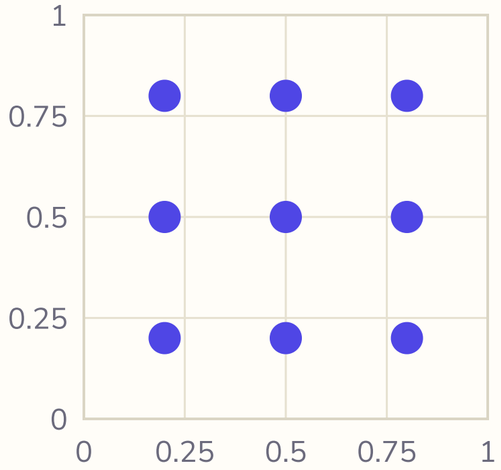
Ejercicio 1: **n_combinaciones = len(busqueda_grid.cv_results_["params"])** y **mejor_auc_grid = busqueda_grid.best_score_**. Con **n_jobs=2** el bosque no tarda demasiado.

REJILLA FRENTE A AZAR · SECCIÓN 1

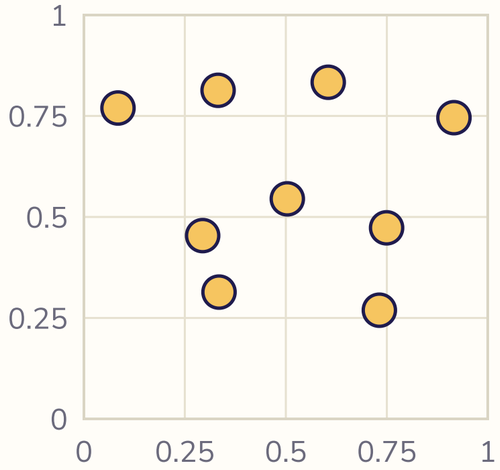
Con 9 pruebas, el azar explora más

Eje horizontal: un hiperparámetro que **importa**. Eje vertical: otro que **apenas influye**.

Rejilla · 3 valores



Al azar · 9 valores



Rejilla

9 pruebas, pero solo **3 valores distintos** del hiperparámetro que importa.

Aleatorio

9 pruebas y **9 valores distintos** del que importa, con el mismo coste.

RandomizedSearchCV prueba n_iter combinaciones al azar: explora más con el mismo presupuesto.

ALEATORIO · EJERCICIO 2 (CASA)

Distribuciones en lugar de listas

Distribución	Qué genera	5 valores de ejemplo
`randint(2, 12)`	Enteros del 2 al **11** (el 12 no entra)	7 · 10 · 11 · 7 · 2
`loguniform(0.001, 1)`	Decimales repartidos por **órdenes de magnitud**	0.0178 · 0.1449 · 0.0010 · 0.0081 · 0.0028
`uniform(0.5, 0.5)`	Decimales de 0,5 a 1,0: **(inicio, ancho)**	0.7085 · 0.8602 · 0.5001 · 0.6512 · 0.5734

Por qué loguniform

Un tercio de las pruebas cae por debajo de 0,01 y otro tercio por encima de 0,1. Con una uniforme, **solo el 1 %** bajaría de 0,01.

Ejercicio 2

`RandomizedSearchCV(pipe_lgb, distribuciones, n_iter=25, cv=cv, scoring="roc_auc", random_state=SEMILLA)`: 25 × 3 = **75** entrenamientos.

Con `subsample` añade `subsample_freq=1` al modelo, o no tendrá efecto.

LEER LOS RESULTADOS · EJERCICIO 3 (CASA)

Muchos empates: elige lo sencillo

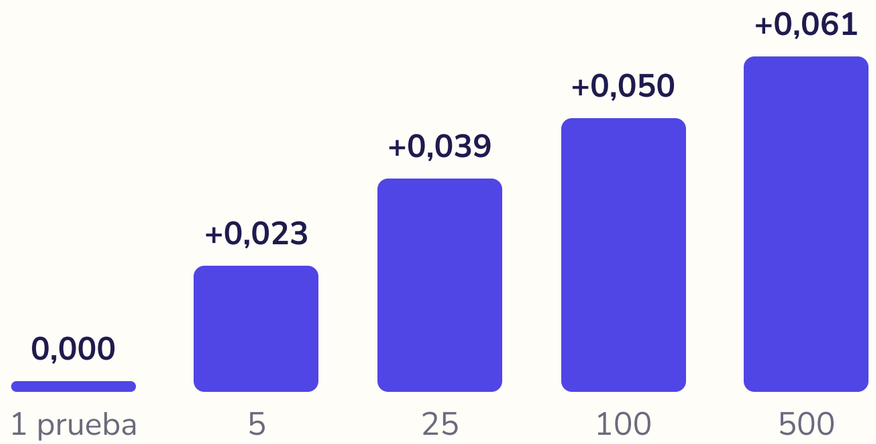
max_depth	min_samples_leaf	mean_test_score	std_test_score	¿Dentro del umbral?
10	21	0.892	0.022	Sí
6	13	0.886	0.027	Sí
6	19	0.884	0.024	Sí
9	10	0.874	0.035	Sí
8	7	0.866	0.036	No
12	6	0.859	0.022	No

pd.DataFrame(busqueda.cv_results_): 8 combinaciones, las 6 mejores. Umbral 0,892 – 0,022 = **0,870**: dentro, **4 de 8**.

Entre las empatadas, la más sencilla: max_depth = 6, no 10. Mismo rendimiento, menos riesgo.

LA NOTA OPTIMISTA · EJERCICIO 4 (CASA)

La mejor nota de validación es en parte suerte



Ejemplo con el árbol

Validación **0.892** → test **0.850**: 0.042 menos.

Qué lo explica

Entre muchas combinaciones, alguna sale bien por azar: la «maldición del ganador».

```
auc_test = roc_auc_score(y_test, busqueda.best_estimator_.predict_proba(X_test)[:, 1])
```

El test se usa una sola vez, al final, con el modelo ya elegido. Nunca para elegir.

MINI-RETO (CASA)

El informe de la búsqueda

Clave del diccionario **informe**	Ejemplo con el árbol
modelo	"arbol"
parametros	{'max_depth': 10, 'min_samples_leaf': 21}
auc_validacion	0.892
auc_test	0.85
n_entrenamientos	8 × 5 = 40
recomendacion	"El AUC en test baja 0,042 y hay 4 empatadas: la nota de validación es optimista."

n_entrenamientos = combinaciones × pliegues (24 con la rejilla del cuaderno, 75 con el aleatorio), sin contar el reentrenado.

Elige el mejor buscador por su AUC en validación, y en la recomendación menciona la diferencia con el test o los empates.

HOY HAS APRENDIDO

Buscar sin hacer trampas

Los botones

Hiperparámetros: los fijas tú antes de entrenar. Los buscadores los prueban con validación cruzada **sobre train**

Dos buscadores

GridSearchCV: todas las combinaciones.
RandomizedSearchCV: **n_iter** al azar, con más valores distintos

Una nota honesta

cv_results_ muestra empates: elige lo **simple**. El test se usa **una sola vez**

Siguiente: paso 3 del reto · día 2 (jueves 12/11)